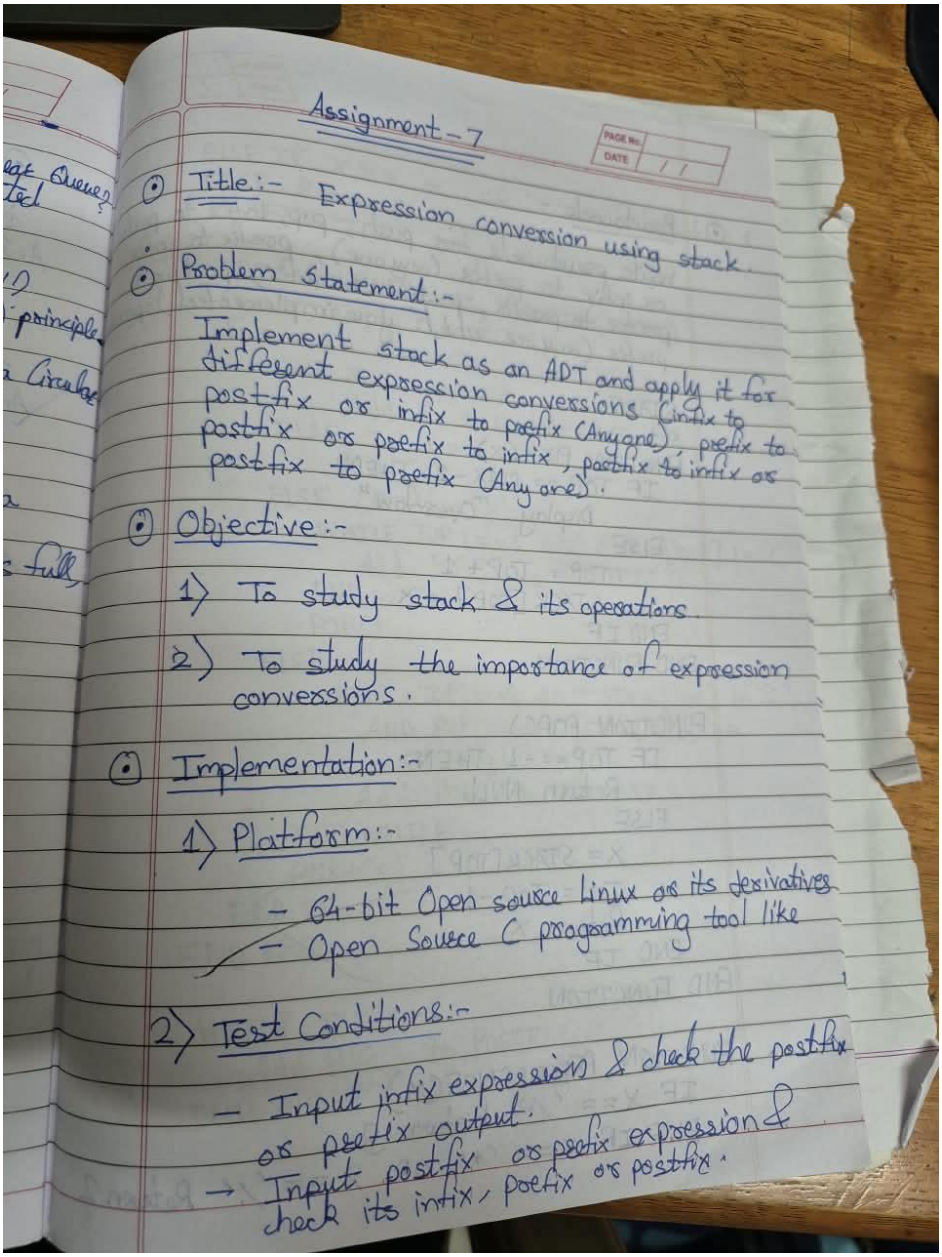
ASSIGNMENT-7

NAME – Abhinav Ajitkumar Chavan

PRN – 1262251966

BATCH - A1



Assignment - 7

PAGE No.
DATE / /

⊙ Title :- Expression conversion using stack.

⊙ Problem Statement :-

Implement stack as an ADT and apply it for different expression conversions (infix to postfix or infix to prefix (Any one), prefix to postfix or prefix to infix, postfix to infix or postfix to prefix (Any one).

⊙ Objective :-

1) To study stack & its operations.

2) To study the importance of expression conversions.

⊙ Implementation :-

1) Platform :-

- 64-bit Open source linux or its derivatives
- Open Source C programming tool like

2) Test Conditions :-

- Input infix expression & check the postfix or prefix output.
- Input postfix or prefix expression & check its infix, prefix or postfix.

PAGE No.
DATE / /

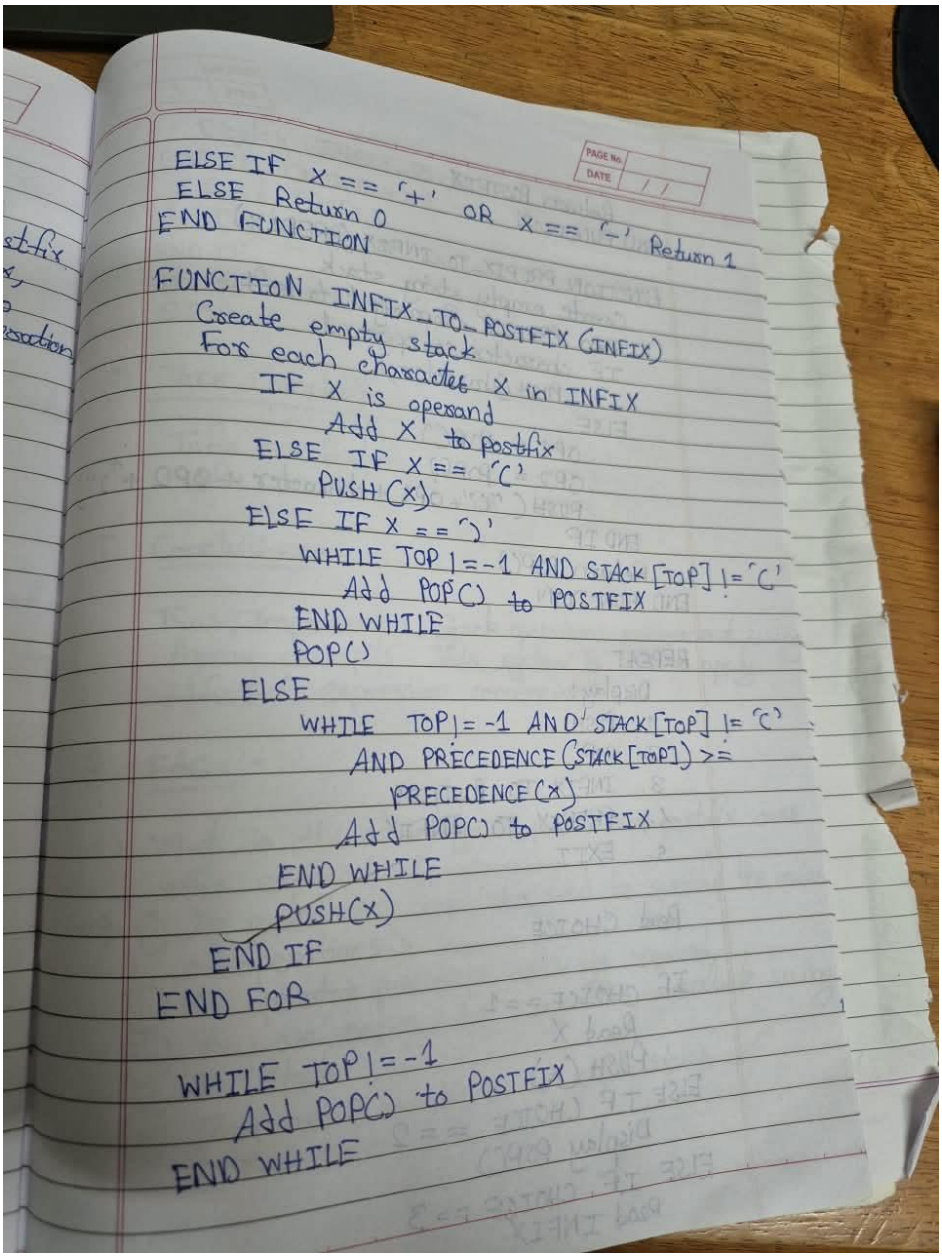
⊙ Pseudocode :-

Write pseudocode for push, pop, infix to postfix or infix to prefix (any one), prefix to infix, prefix to postfix, postfix to infix, postfix to prefix (any one which you implemented) operation

```
START
Set TOP=-1
FUNCTION PUSH(X)
  IF TOP == MAX-1 THEN
      Display "Overflow"
  ELSE
      TOP = TOP+1
      STACK[TOP] = X
  END IF
END FUNCTION

FUNCTION POP()
  IF TOP==-1 THEN
     Return NULL
  ELSE
     X = STACK[TOP]
     TOP = TOP-1
     Return X
  END IF
END FUNCTION

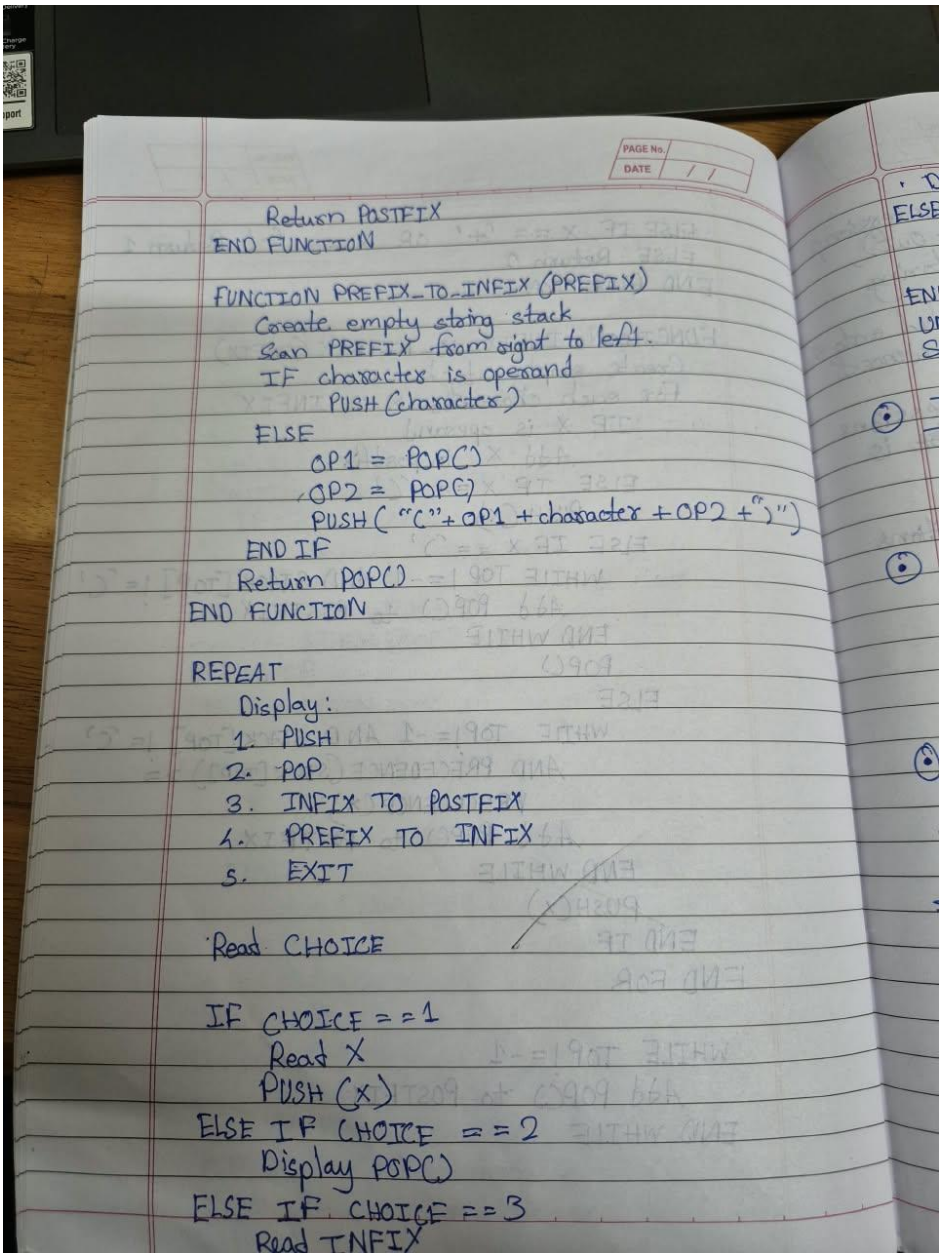
FUNCTION PRECEDENCE(X)
  IF X == '^' Return 3
  ELSE IF X == '*' OR X == '/' Return 2
```



```
ELSE IF X == '+' OR X == '-' Return 1
ELSE Return 0
END FUNCTION

FUNCTION INFIX_TO_POSTFIX (INFIX)
  Create empty stack
  For each character X in INFIX
    IF X is operand
      Add X to postfix
    ELSE IF X == '('
      PUSH (X)
    ELSE IF X == ')'
      WHILE TOP != -1 AND STACK[TOP] != '('
        Add POP() to POSTFIX
      END WHILE
      POP()
    ELSE
      WHILE TOP != -1 AND STACK[TOP] != '('
        AND PRECEDENCE(STACK[TOP]) >=
          PRECEDENCE(X)
        Add POP() to POSTFIX
      END WHILE
      PUSH(X)
  END IF
END FOR

WHILE TOP != -1
  Add POP() to POSTFIX
END WHILE
```



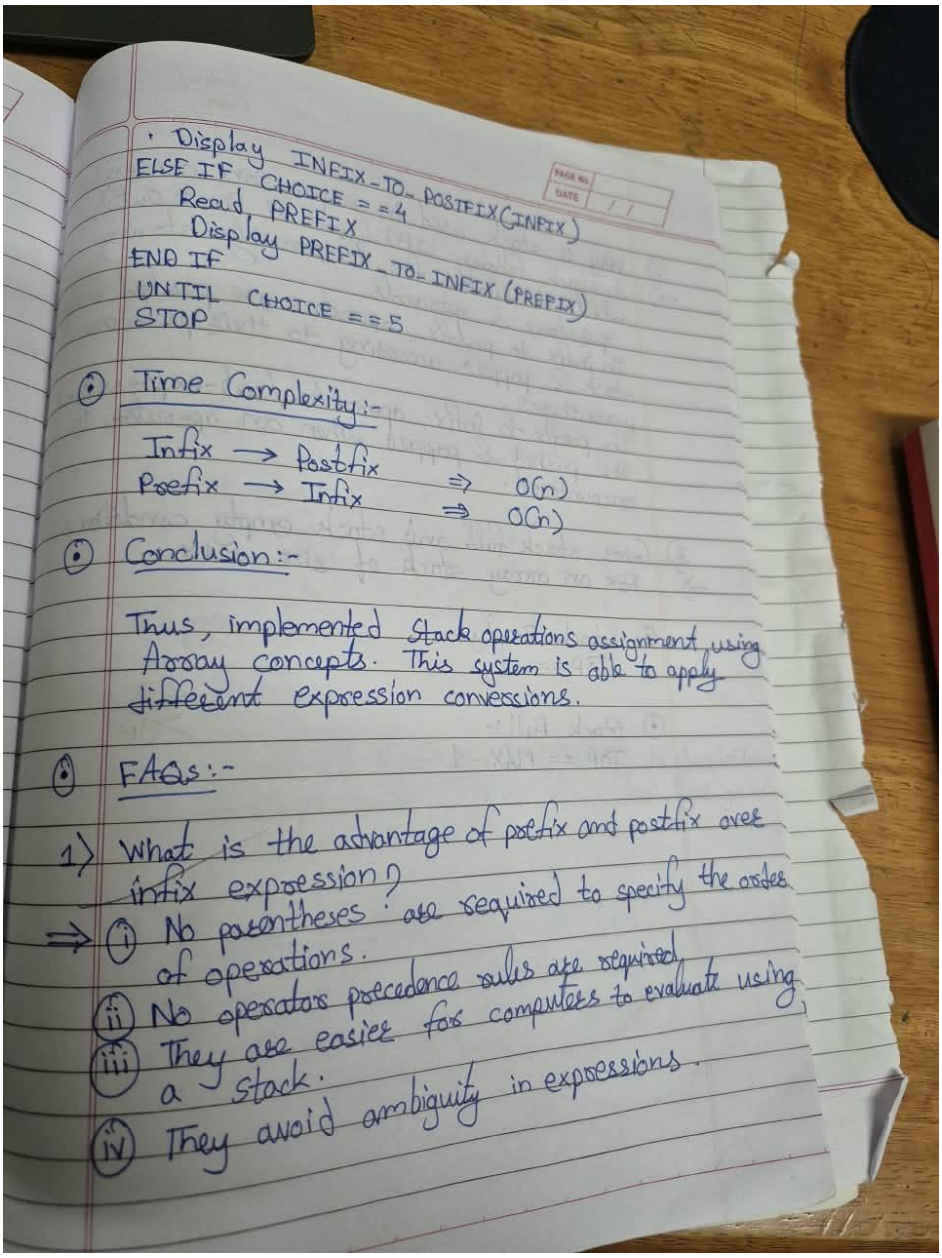
```
    Return POSTFIX
END FUNCTION

FUNCTION PREFIX_TO_INFIX (PREFIX)
    Create empty string stack
    Scan PREFIX from right to left.
    IF character is operand
        PUSH (character)
    ELSE
        OP1 = POP()
        OP2 = POP()
        PUSH ( "(" + OP1 + character + OP2 + ")" )
    END IF
    Return POP()
END FUNCTION

REPEAT
    Display:
    1. PUSH
    2. POP
    3. INFIX TO POSTFIX
    4. PREFIX TO INFIX
    5. EXIT

    Read CHOICE

    IF CHOICE == 1
        Read X
        PUSH (X)
    ELSE IF CHOICE == 2
        Display POP()
    ELSE IF CHOICE == 3
        Read INFIX
```



• Display INFIX_TO_POSTFIX(INFIX)
ELSE IF CHOICE == 4
Read, PREFIX
Display PREFIX_TO_INFIX (PREFIX)
END IF
UNTIL CHOICE == 5
STOP

⊙ Time Complexity:-

Infix → Postfix ⇒ O(n)
Postfix → Infix ⇒ O(n)

⊙ Conclusion:-

Thus, implemented Stack operations assignment using Array concepts. This system is able to apply different expression conversions.

⊙ FAQs:-

1) What is the advantage of postfix and postfix over infix expression?

⇒ (i) No parentheses are required to specify the order of operations.
(ii) No operator precedence rules are required.
(iii) They are easier for computers to evaluate using a stack.
(iv) They avoid ambiguity in expressions.

PAGE No.
DATE / /

2) Why is stack used in expression conversion?
⇒ A stack follows LIFO (Last IN, First out), which is suitable for temporarily storing operators & operands.
In infix to postfix, operators are pushed onto the stack & popped according to their precedence & parentheses.
In prefix to infix, operands / sub-expressions are pushed & popped when an operator is encountered.

3) Give stack full and stack empty conditions.
⇒ For an array stack of size MAX :-

- Stack Empty :-
  TOP == -1

- Stack Full :-
  TOP == MAX - 1